



Kaggle Benchmarks

Building and sharing the future of AI evaluation
through open community benchmarks.

Ivan Leo , ileo@google.com, kaggle.com/benchmarks

May 2026

Benchmarks

- Kaggle aims to be the world's trusted and open platform for evaluating AI on tough tasks that drive progress
- Since Jan 2026, we've had over **8K** evals measuring everything from mathematical reasoning to social cognition
- We've recently launched community benchmarks where anyone can create and share their own benchmarks for free

01

Local Benchmarks

Define and build your benchmarks directly as code



Setup & Workflow

Bring your favourite code editor (Eg. Antigravity) and run a single command to start building, testing and deploying your benchmarks on Kaggle

AGENTIC WORKFLOW

```
npx skill add kaggle-skills/kaggle-create-benchmark
```

Defining a Task

Start by defining evaluation logic for a single test case.

- **The Task Decorator:** Annotate a function with `@kbench.task(store_task=False)` to define an atomic evaluation block.
- **Model Proxy:** Use `llm.prompt()` to query the model under test securely.
- **Assertions:** Validate model outputs with built-in assertion helpers (`assert_true`) for precise tracking.

SINGLE TEST CASE

```
import kaggle_benchmarks as kbench

@kbench.task(store_task=False)
def single_qa_task(llm, question, answer) -> (
    prompt = f"{question}\n\nPut answer in <ar

    # Prompt the proxy
    response = llm.prompt(prompt)
    predicted = extract_answer(response)

    # Validate correctness
    is_correct = (str(predicted) == str(answer))
    kbench.assertions.assert_true(
        is_correct,
        expectation=f"Expected {answer}, got {predicted}"
    )

    return {"is_correct": is_correct}
```

Scaling to Datasets

Map your single test case over many potential inputs concurrently.

- **Scale Seamlessly:** Define your benchmark loop and use `.evaluate()` to map the atomic task over a Pandas DataFrame.
- **Concurrent Execution:** `llm.prompt()` automatically scales evaluation, letting you run multiple models against the benchmark simultaneously.
- **Parallel Jobs:** Use `n_jobs` to accelerate evaluation across rows.

DATASET EVALUATION

```
import pandas as pd

# 1. Simple DataFrame format
df = pd.DataFrame([
    {"question": "What is 2 + 2?", "answer": '2'},
    {"question": "What is 3 * 5?", "answer": '15'}
])

# 2. Dataset Evaluation Task
@kbench.task()
def multi_qa_task(llm, dataset) -> float:
    runs = single_qa_task.evaluate(
        evaluation_data=dataset,
        llm=[llm],
        n_jobs=4
    )
    eval_df = runs.as_dataframe()
    return float(eval_df.result.str.get("is_co
```

Testing & Publishing

Execute real-time evaluations from your terminal.

- **Local Execution:** Run the task locally using Gemini Flash
- **Real-time Traces:** Show execution traces and assertion outputs directly in the terminal
- **Multi-Model Execution:** Pass multiple models to evaluate them side-by-side on Kaggle's infrastructure.
- **Publish:** Move local task to a live Kaggle Benchmark page and leaderboard

END-TO-END WORKFLOW

```
# 1. Initialize environment & get proxy auth
kaggle b init -y

# 2. Push local file to a Kaggle Benchmark
kaggle b t push my-task -f task.py --wait

# 3. Run concurrently against multiple models
kaggle b t run my-task -m google/gemini-3.1-f

# 4. Download evaluation outputs
kaggle b t download my-task -o ./results
```

02

Early Access Program

Help us build the future of agentic evaluation.



Join the Waitlist

We are recruiting early-access testers!

- **What you get:** Access to the `kaggle-`
`benchmarks` library with new local development support.
- **CLI extensions:** The new CLI extensions for local task management.
- **Skill File:** The `skill.md` automation file for AI coding agents.
- **Future Roadmap:** Self-service Docker-backed tasks for complex, agentic Benchmarks.

Scan to join the waitlist



Closing Thoughts

"Progress in AI shouldn't be defined by a small number of large labs. It should be built, verified, and shared by the community."

Join us: [Waitlist Google Form](#)